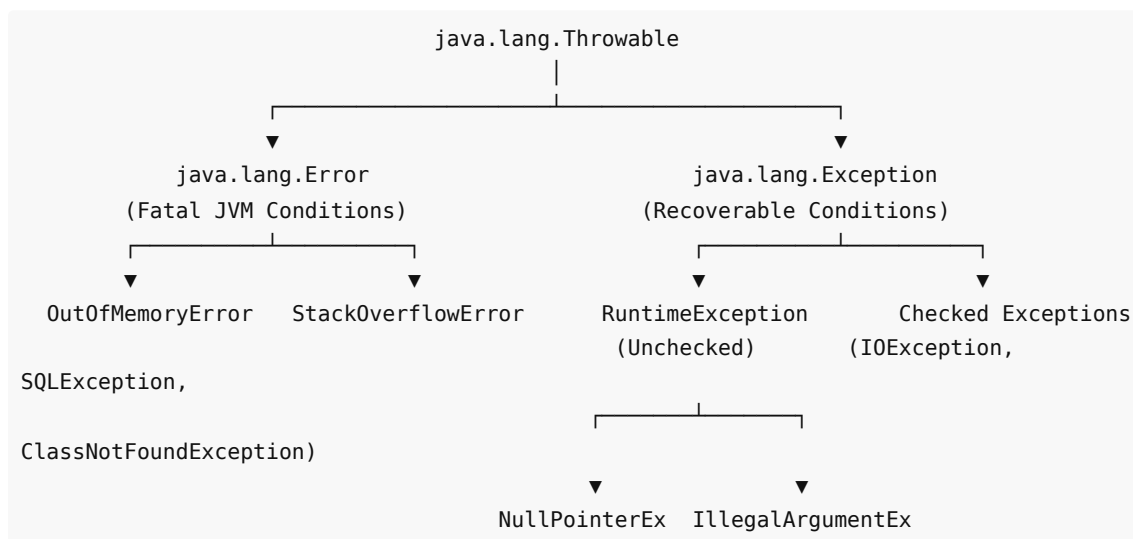


SECTION 7 — EXCEPTIONS AND ERROR HANDLING (Senior SDET Masterclass)

Topics Covered

- 7.1 The Throwable Hierarchy (Error vs. Exception vs. RuntimeException)
- 7.2 Checked vs. Unchecked Exceptions (Architectural Design Philosophy)
- 7.3 try / catch / finally Execution Mechanics & JVM Control Flow
- 7.4 Multi-Catch & Union Types (Java 7+ Syntax & Bytecode Constraints)
- 7.5 Try-With-Resources & AutoCloseable (Suppressed Exceptions Internals)
- 7.6 throw vs. throws (Method Signatures & Stack Frame Unwinding)
- 7.7 Enterprise Custom Framework Exception Hierarchies (Categorized Error Codes)
- 7.8 Exception Propagation & Stack Trace Anatomy
- 7.9 final vs. finally vs. finalize() (The Classic Senior Distinction)
- 7.10 Top 10 Automation Exceptions in Java (Root Causes & Resilient Fixes)
- 7.11 Best Practices & Catastrophic Anti-Patterns in Test Suites
- 7.12 Chained Exceptions & Root Cause Preservation
- 7.13 High-Stakes Senior Exception Interview Questions & Spoken Solutions

7.1 The Throwable Hierarchy



The 3 Core Branches of Throwable

1. **java.lang.Error** : Represents serious, fatal conditions that a reasonable application should **never attempt to catch** (e.g. `OutOfMemoryError` , `StackOverflowError` , `VirtualMachineError`). They signal unrecoverable JVM or OS-level resource exhaustion.
2. **Checked Exception** : Extends `Exception` directly (excluding `RuntimeException`). The compiler **mandates** that they be declared in a `throws` clause or handled in a `try-catch` block. Represents anticipated recoverable external failures (e.g., file not found, network socket disconnect).
3. **Unchecked RuntimeException** : Extends `RuntimeException` . Represents programming defects, logic bugs, or API contract violations (e.g. `NullPointerException` , `IllegalArgumentException`).

`IndexOutOfBoundsException` , `IllegalArgumentException`). The compiler does not require explicit handling.

7.2 Checked vs. Unchecked Exceptions in Modern Framework Design

The Modern Architectural Consensus

- **Legacy Java:** Advocated checked exceptions for any recoverable scenario.
 - **Modern Frameworks (Spring, Playwright, Selenium, TestNG):** Rely almost exclusively on **Unchecked (RuntimeException)**.
 - **Why?** Checked exceptions clutter business code with boilerplate, leak internal implementation details through method signatures (e.g., declaring `throws SQLException` in a high-level `login()` method), and break lambda expressions and functional pipelines.
-

7.3 try / catch / finally & 7.5 Try-With-Resources

1. finally Block Guarantees & Caveats

The `finally` block is **always guaranteed to execute**, regardless of whether an exception was thrown or caught, even if the `try` block contains a `return` statement.

- **The Only 2 Scenarios Where finally Does NOT Execute:**
 1. `System.exit(0)` is invoked explicitly.
 2. The JVM crashes catastrophically (`OutOfMemoryError` , `SIGKILL` from OS).

*[!WARNING] **Never return a value from inside a finally block!** Returning from `finally` silently swallows and discards any exception thrown in the preceding `try` block, masking fatal bugs!*

2. Try-With-Resources & Suppressed Exceptions (AutoCloseable)

Introduced in Java 7, any object implementing `java.lang.AutoCloseable` or `java.io.Closeable` can be instantiated inside `try (...)` .

- The compiler automatically invokes `.close()` upon exiting the block, in **reverse order of declaration**, even if exceptions occur.
- **Suppressed Exceptions:** If both the `try` block and the `.close()` method throw exceptions, the exception from `try` is preserved as the primary exception, and the closure exception is attached to it via `e.addSuppressed()` . Retrieve it using `e.getSuppressed()` .

```
package com.deloitte.sdet.io;

import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;

public final class SafeResourceReader {

    public static String readFirstTestRecord(String filePath) throws IOException {
        // Automatically manages stream lifecycle and guarantees closure
    }
}
```

```

        try (BufferedReader reader = new BufferedReader(new FileReader(filePath))) {
            return reader.readLine();
        }
    }
}

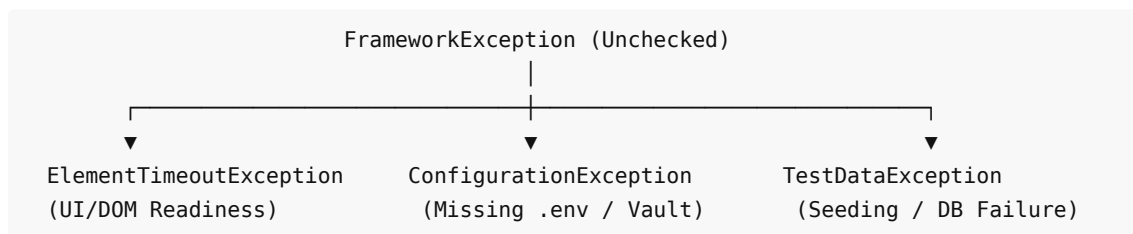
```

7.7 Enterprise Custom Framework Exception Hierarchies

In a 5,000+ test enterprise framework, tests fail for three distinct reasons:

1. **Application Bug (Valid Defect)**
2. **Automation Code Defect (Script Bug)**
3. **Infrastructure / Environment Instability (Network, Grid, Auth Provider)**

A senior architect designs a categorized exception hierarchy so test listeners and CI pipelines can automatically classify failures without manual triage.



Production-Ready Code: Structured Framework Exception Hierarchy

```

package com.deloitte.sdet.exceptions;

public class FrameworkException extends RuntimeException {

    private final String errorCode;

    public FrameworkException(String errorCode, String message) {
        super(String.format("[%s] %s", errorCode, message));
        this.errorCode = errorCode;
    }

    public FrameworkException(String errorCode, String message, Throwable cause) {
        super(String.format("[%s] %s", errorCode, message), cause);
        this.errorCode = errorCode;
    }

    public String getErrorCode() { return errorCode; }
}

// Subclass 1: Infrastructure and Wait Failures
public class ElementTimeoutException extends FrameworkException {
    public ElementTimeoutException(String message, Throwable cause) {
        super("ERR-TIMEOUT-001", message, cause);
    }
}

```

```
// Subclass 2: Missing Test Data or Collisions
public class TestDataCollisionException extends FrameworkException {
    public TestDataCollisionException(String message) {
        super("ERR-DATA-002", message);
    }
}
```

7.9 final vs. finally vs. finalize()

Keyword	Category	Functionality
final	Modifier	Declares immutable variables, un-overridable methods, or un-extendable classes.
finally	Control Flow Block	Ensures execution of cleanup code following a try-catch block.
finalize()	Method in Object	Deprecated since Java 9, removed in Java 18+. Unreliable GC callback. Replaced by <code>java.lang.ref.Cleaner</code> .

7.10 Top 10 Automation Exceptions in Java

Exception	Root Cause	Senior Architectural Resolution
StaleElementReferenceException	DOM node was removed, replaced, or updated via AJAX/SPA re-render after lookup.	Encapsulate locator inside Page Object method so it re-queries on every interaction; avoid caching WebElement fields; use Playwright auto-re-resolving locators.
NoSuchElementException	Element absent from DOM when findElement() executed.	Replace raw findElement() with explicit wait ExpectedConditions.visibilityOfElementLocated().
ElementClickInterceptedException	Element exists, but an overlay (spinner, backdrop, sticky banner) obscures its coordinates.	Wait for overlay to become invisible (invisibilityOfElementLocated) before clicking, or scroll into view.

TimeoutException	Explicit wait condition was not satisfied within the configured duration.	Inspect network latency; increase timeout conditionally; verify locator selector is correct.
NoSuchSessionException	Browser crashed, closed, or driver instance was quit by another parallel thread.	Ensure <code>ThreadLocal<WebDriver></code> isolation; eliminate static driver variables.
NullPointerException	Uninitialized Page Object, unboxed null wrapper, or missing configuration key.	Apply <code>Objects.requireNonNull()</code> , use constructor injection, avoid autoboxing null wrappers.
ConcurrentModificationException	Modifying a collection while traversing it using an iterator.	Use <code>CopyOnWriteArrayList</code> , <code>ConcurrentHashMap</code> , or <code>Iterator.remove()</code> .
IllegalStateException	WebDriver executable path not found, or driver invoked after <code>.quit()</code> .	Use <code>WebDriverManager</code> or modern Selenium 4 built-in Selenium Manager.
InvalidSelectorException	Syntactically malformed XPath or CSS selector.	Validate selector syntax; avoid unescaped single quotes in XPath strings.
JsonPathException	REST Assured failed to parse malformed JSON	Validate response Content-Type and status code before attempting JSON parsing.

	response or invalid path.	
--	------------------------------	--

7.11 Best Practices & Catastrophic Anti-Patterns

Anti-Pattern 1: Swallowing Exceptions

```
// CATASTROPHIC ANTI-PATTERN
try {
    submitOrder();
} catch (Exception e) {
    // SILENT SWALLOW: Test passes as GREEN even though checkout failed!
}
```

Fix: Always rethrow as a custom `FrameworkException` or fail the assertion explicitly:

```
try {
    submitOrder();
} catch (Exception e) {
    throw new FrameworkException("CHECKOUT_FAILED", "Order submission failed", e);
}
```

Anti-Pattern 2: Catching `Throwable`

```
// NEVER DO THIS
catch (Throwable t) { ... }
```

Why: Catches fatal JVM errors like `OutOfMemoryError` or `StackOverflowError`, masking catastrophic infrastructure collapse and preventing clean container shutdowns.

Anti-Pattern 3: Losing the Original Cause

```
// WRONG: Erases original stack trace
catch (IOException e) {
    throw new RuntimeException("File read error: " + e.getMessage());
}

// CORRECT: Chained Exception preserves root cause
catch (IOException e) {
    throw new RuntimeException("File read error", e);
}
```

7.13 High-Stakes Senior Exception Interview Questions & Spoken Solutions

Q1: "How does a Senior SDET design an exception strategy to distinguish an Application Defect from a Flaky Infrastructure Failure in CI reports?"

"In enterprise automation frameworks, we design a two-tiered exception architecture:

1. All assertions (e.g. `TestNG Assert` or `AssertJ`) throw `java.lang.AssertionError`, which inherits directly from `Error`, NOT `Exception`. In our custom `ITestListener`, any failure where `result.getThrowable()` instanceof `AssertionError` is classified as an **Application Business Defect**.
2. All framework infrastructure, driver, wait, and database issues throw our custom `FrameworkException` (inheriting from `RuntimeException`). Subclasses include `InfrastructureException`, `ElementTimeoutException`, and `TestDataException`.
3. Our CI listener inspects the failure hierarchy. If the exception is a `FrameworkException` with error code `ERR-TIMEOUT`, it is routed to our automated diagnostic retry analyzer with trace capture. If it is an `AssertionError`, it is immediately logged as a high-confidence defect ticket in Jira without wasteful retries."

Q2: "What is the difference between `ClassNotFoundException` and `NoClassDefFoundError` ?"

"The distinction centers on compile-time vs. runtime lifecycle:

- `ClassNotFoundException` is a **checked Exception** that occurs at runtime when an application explicitly tries to load a class via reflection (e.g. `Class.forName("com.mysql.cj.jdbc.Driver")` or `ClassLoader.loadClass()`) and the class file cannot be located on the classpath.
- `NoClassDefFoundError` is an **unchecked Error** that occurs when a class was **successfully present at compile time**, but at runtime, the JVM attempts to reference the class (via `new` or method call) and the class definition is missing from the classpath. This typically happens in Maven builds due to transitive dependency conflicts, excluded JARs, or static initialization failures (`<clinit>` threw an exception when the class was first loaded)."